Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина «Модели данных и системы управления базами данных»

ОТЧЕТ
к лабораторной работе №2
на тему:
«ТРИГГЕРЫ»
БГУИР 6-05-0612-02 67

Выполнил студент группы 353503
КОХАН Артём Игоревич

_____
(дата, подпись студента)

Проверил ассистент каф. информатики
ИОВИЧ Евгения Александровна

_____
(дата, подпись преподавателя)

Минск 2026

1 ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ

1 Построить две таблицы STUDENTS и GROUPS реализующих соответственно справочник студентов и справочник групп:

Таблица 1.1 – Таблицы STUDENTS и GROUPS

Поле	Тип	Комментарий
STUDENTS		
ID	Number	Код студента
NAME	VARCHAR2	Имя студента
GROUP_ID	Number	Код группы
GROUPS		
ID	Number	Код группы
NAME	VARCHAR2	Название группы
C_VAL	Number	Количество студентов в группе

2 Реализовать триггеры для таблиц задания 1: проверку целостности (проверка на уникальность полей ID), генерацию автоинкрементного ключа и проверку уникальности для поля GROUP.NAME.

3 Реализовать триггер, реализующий Foreign Key с каскадным удалением между таблицами STUDENTS и GROUPS.

4 Реализовать триггер, реализующий журналирование всех действий над данными таблицы STUDENTS

5 Исходя из данных предыдущей задачи, реализовать процедуру для восстановления информации на указанный временной момент и на временное смещение.

6 Реализовать триггер, который в случае изменения данных в таблице STUDENTS будет соответственно обновлять информацию C_VAL таблицы GROUPS.

2 КРАТКИЕ ТЕОРИТИЧЕСКИЕ СВЕДЕНИЯ

Триггеры (Triggers) — это специальные хранимые процедуры, которые автоматически выполняются при наступлении определенного события в базе данных. В Oracle PL/SQL триггеры могут срабатывать до или после выполнения операций DML (INSERT, UPDATE, DELETE), а также при выполнении DDL операций или системных событий.

Основные характеристики триггеров:

1 Выполняются автоматически при события.

2 Не могут принимать параметры.

3 Выполняются в контексте транзакции.

4 Могут быть уровня строки (FOR EACH ROW) или уровня оператора.

Типы триггеров DML:

1 BEFORE – выполняется до выполнения операции.

2 AFTER – выполняется после выполнения операции.

3 INSTEAD OF – выполняется вместо операции.

Особенности триггеров уровня строки:

1 Имеют доступ к старым (:OLD) и новым (:NEW) значениям.

2 Срабатывают для каждой изменяемой строки.

3 Могут изменять значения в :NEW для триггеров BEFORE.

Триггеры представляют собой мощный инструмент для автоматизации бизнес-логики и поддержания целостности данных. Они позволяют выполнять сложные проверки и преобразования данных непосредственно на уровне базы данных, что повышает надежность и производительность приложений. Триггеры уровня строки особенно полезны для реализации каскадных операций и ведения детального аудита изменений, так как они имеют доступ к конкретным значениям изменяемых строк.

3 ВЫПОЛНЕНИЕ РАБОТЫ

1 Создание таблиц STUDENTS и GROUPS для хранения информации о студентах и учебных группах.

Листинг 3.1 – Создание таблиц STUDENTS и GROUPS

```
CREATE TABLE groups (
    id NUMBER NOT NULL,
    name VARCHAR2(50) NOT NULL,
    count_val NUMBER DEFAULT 0
);

CREATE TABLE students (
    id NUMBER NOT NULL,
    student_name VARCHAR2(50) NOT NULL,
    group_id NUMBER
);
```

2 Создание триггеров для обеспечения целостности данных и автоматической генерации ключей:

Триггер GROUPS_AUTO_INC_BEFORE_INSERT обеспечивает автоматическую генерацию идентификатора для таблицы GROUPS, если значение не было указано явно. Триггер срабатывает перед вставкой и вычисляет максимальный существующий ID, увеличивая его на 1. Результат работы триггера продемонстрирован на рисунке 3.1.

Триггер STUDENTS_AUTO_INC_BEFORE_INSERT выполняет аналогичную функцию для таблицы STUDENTS, гарантируя уникальность идентификаторов студентов.

	ID	NAME	COUNT_VAL
1	1	iitp_253503	2
2	2	iitp_453503	0
3	3	iitp_353503	0

Рисунок 3.1 – Автоматическая генерация на примере таблицы GROUPS

Триггер GROUPS_CHECK_UNIQUE обеспечивает проверку уникальности как ID группы, так и её названия. При попытке вставить или обновить запись с дублирующимся ID или названием группы генерируется исключение с соответствующим сообщением об ошибке. Попробовав произвести вставку названия группы, которое уже имеется в таблице, получим ожидаемое сообщение об ошибке, что наглядно видно на рисунке 3.2.

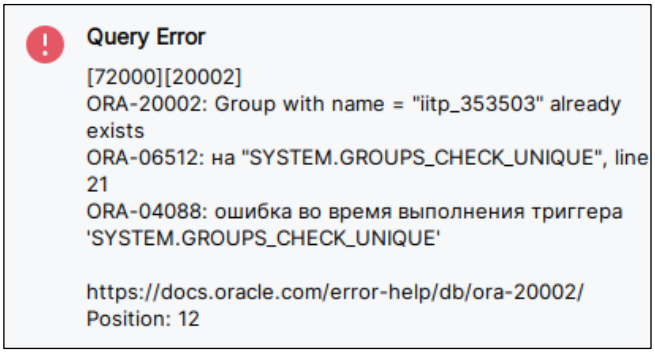

Рисунок 3.2 – Результат работы триггера GROUPS_CHECK_UNIQUE

Триггер STUDENTS_CHECK_UNIQUE контролирует уникальность ID студентов, предотвращая дублирование идентификаторов при операциях вставки и обновления.

3 Триггер для реализации каскадного удаления. При удалении группы из таблицы GROUPS триггер GROUPS_CASCADE_DELETE автоматически удаляет всех студентов, принадлежащих к этой группе, поддерживая целостность данных. Результат работы тяжело продемонстрировать скринами, но код достаточно прост для понимания (листинг 3.1).

Листинг 3.2 — Триггер каскадного удаления

```
CREATE OR REPLACE TRIGGER GROUPS_CASCADE_DELETE
BEFORE DELETE ON GROUPS
FOR EACH ROW
BEGIN
    DELETE FROM STUDENTS WHERE GROUP_ID = :OLD.ID;
END;
```

4 Создание таблицы журналирования и триггера для отслеживания всех изменений в таблице STUDENTS. Таблица STUDENTS_LOGS содержит информацию о типе операции (INSERT, UPDATE, DELETE), старых и новых значениях, а также метку времени и пользователя, выполнившего изменение. При выполнениях каждой из операций (INSERT, UPDATE, DELETE) с таблицей STUDENTS происходит добавление в таблицу STUDENTS. Результат работы таблицы и триггера представлен на рисунке 3.3.

	AUDIT_ID	STUDENT_ID	ACTION_TYPE	OLD_NAME	NEW_NAME	OLD_GROUP_ID	NEW_GROUP_ID
1	1	3	INSERT	<null>	Artsiom	<null>	1
2	2	4	INSERT	<null>	Artsiom	<null>	1
3	3	1	UPDATE	Artsiom	Vasya	1	1
4	4	2	UPDATE	Artsiom	Vasya	1	1
5	5	3	UPDATE	Artsiom	Vasya	1	1
6	6	4	UPDATE	Artsiom	Vasya	1	1
7	7	1	DELETE	Vasya	<null>	1	<null>
8	8	2	DELETE	Vasya	<null>	1	<null>
9	9	3	DELETE	Vasya	<null>	1	<null>
10	10	4	DELETE	Vasya	<null>	1	<null>

Рисунок 3.3 – Заполнение таблицы STUDENTS_LOGS триггером

5 Процедуры восстановления данных на основе журнала изменений. Процедура RESTORE_TO_TIMESTAMP позволяет восстановить состояние таблицы STUDENTS на указанный момент времени, анализируя записи в журнале. Процедура RESTORE_BY_OFFSET восстанавливает состояние на указанное количество минут назад от текущего времени. Пример вызова процедур и результат их работы представлен на рисунке 3.4.

```
[2026-01-25 01:52:41] SYSTEM> CALL RESTORE_TO_TIMESTAMP(TIMESTAMP '2026-01-24 17:00:00')
[2026-01-25 01:52:41] completed in 3 ms
Students at the 24.01.2026 17:00:00:
ID: 3, name: Artsiom, group: 1
ID: 4, name: Artsiom, group: 1
[2026-01-25 01:52:46] SYSTEM> CALL RESTORE_BY_OFFSET(15)
[2026-01-25 01:52:46] completed in 2 ms
Students at the 24.01.2026 22:37:46:
ID: 1, name: Artsiom, group: 1
ID: 2, name: Artsiom, group: 1
```

Рисунок 3.4 – Вызов процедур восстановления данных

6 Триггер для автоматического обновления счетчика студентов в группах. Триггер UPDATE_GROUPS_COUNT_VAL срабатывает после любых изменений в таблице STUDENTS и пересчитывает количество студентов в каждой группе, обновляя поле COUNT_VAL в таблице GROUPS.

Листинг 3.3 – Триггер для автоматического обновления счетчика студентов

```
CREATE OR REPLACE TRIGGER UPDATE_GROUPS_COUNT_VAL
AFTER INSERT OR UPDATE OR DELETE ON STUDENTS
BEGIN
    FOR grp IN (SELECT id FROM groups) LOOP
        UPDATE groups
        SET count_val = (
            SELECT COUNT(*)
            FROM students s
            WHERE s.group_id = grp.id
        )
        WHERE groups.id = grp.id;
    END LOOP;
END UPDATE_GROUPS_COUNT_VAL;
```

В результате любых изменений наблюдаем изменение значения COUNT_VAL в таблице GROUPS, что можно заметить на рисунке 3.5.

	ID	NAME	COUNT_VAL
1	1	iitp_253503	2
2	2	iitp_453503	0
3	3	iitp_353503	0

Рисунок 3.5 – Автоматическое обновления счетчика студентов

ВЫВОД

В ходе выполнения лабораторной работы были успешно изучены и применены на практике механизмы триггеров в Oracle PL/SQL. Были созданы триггеры различных типов и назначений, демонстрирующие возможности автоматизации обработки данных в СУБД. Разработаны триггеры для автоматической генерации первичных ключей, что обеспечивает уникальность идентификаторов. Реализована проверка уникальности данных на уровне триггеров, включая проверку уникальности названий групп, что дополняет стандартные ограничения целостности. Создан механизм каскадного удаления через триггер, обеспечивающий поддержание ссылочной целостности между таблицами STUDENTS и GROUPS. Разработана система журналирования всех изменений в таблице STUDENTS с сохранением полной истории изменений, включая старые и новые значения, тип операции, пользователя и время изменения. Реализованы процедуры восстановления данных на произвольный момент времени на основе журнала изменений, что даёт возможность просмотра логов и отката изменений. Создан триггер для поддержания автоматического обновления счетчика студентов в группах при любых изменениях с таблицей студентов.

Все триггеры корректно обрабатывают различные сценарии работы с данными, включая вставку, обновление и удаление записей. Полученные знания и навыки позволяют эффективно использовать триггеры для реализации сложной бизнес-логики, обеспечения целостности данных в системах управления базами данных Oracle.